LINUX SHELL SCRIPTING / LAB PROJECT REPORT

Event RSVP Manager

L37 | College Fest Guest List & Invitation Management

STUDENT	ROLL / DIVISION	PROJECT ID
HAMZA VORA	F48 / G	L37

Problem Statement

Develop a menu-driven Bash application that tracks guests, records RSVP decisions, summarises food choices, generates personal invitation files, and finds guest records for a college fest organising committee.

Project requirements

Requirement	Implementation
Data	data/guests.csv with 36 fictional guests (30 minimum)
Main commands	here-documents, awk, grep, sort, uniq -c
Five functions	Add guest; RSVP counts; food summary; invitations; search
Scheduled task	At 6:00 PM, write updated counts to reports/rsvp_status.txt
Utilities	Menu, view data, tar.gz backup, activity log, input validation

Folder structure

```
rsvpdesk/
  main.sh                menu and command-line options
  lib/functions.sh       core Bash functions
  data/guests.csv        36 fictional guest records
  data/event.conf        editable event details
  invitations/           generated guest letters
  reports/ backups/ logs/ scheduled reports, backups, logs
  tests/run_tests.sh     isolated regression tests
```

Scope note: Guest records and event information are demonstrative. No invitation emails are sent. Actual event details and participant consent must be supplied by the committee.

REQUIRED FEATURES / 01–03

Guest records and analytics

01. Add a guest

The application reads four fields with read, checks for empty names and invalid emails, enforces yes/no/maybe and food categories, and rejects duplicate emails. A successful entry appends a complete CSV row.

```
Demo input: Demo Guest | demo.guest@example.com | yes | veg
== Add a guest ==
Guest added: Demo Guest (demo.guest@example.com)
```

02. RSVP counts (yes / no / maybe)

The system reads the CSV status column, counts each RSVP using awk, and independently lists frequencies with sort | uniq -c.

```
== RSVP status report ==
Total guests: 37
RSVP            COUNT
------------ --------
yes                19
no                  8
maybe              10
Frequency check: 10 maybe | 8 no | 19 yes
```

03. Food preference summary

Counts each dietary choice for all guests and separately for guests who confirmed yes. The confirmed column provides a more sensible initial catering estimate.

```
== Food preference summary ==
PREFERENCE        ALL     RSVP=YES
------------ -------- ------------
veg                15           12
nonveg              6            3
jain                8            4
none                8            0
```

Demonstration evidence: These transcripts were captured from an automated Linux test copy after adding one guest. They are not screenshots of your friend's WSL session.

REQUIRED FEATURES / 04–05

Invitations and guest search

04. Generate invitations using here-documents

Menu option 4 supports a single guest, confirmed attendees, or the full list. For each selected record, a Bash here-document writes a personalised .txt invitation in the invitations folder. Invitation files are not emailed.

```
Example generated file: invitations/invite_demo.guest_example.com.txt
==================================================
               EVENT INVITATION
==================================================
Dear Demo Guest,
You are invited to join us for:
  Event   : College Fest (Demo)
  Date    : To be announced
  Time    : To be announced
  Venue   : Campus Auditorium
Guest email      : demo.guest@example.com
Current RSVP     : yes
Food preference  : veg
Warm regards, College Fest Organising Committee
```

05. Search a guest

The user supplies part of a name or email. The script applies grep -iF for case-insensitive, literal matching, then prints the relevant guest fields in a clear format.

```
== Search a guest ==
Name or email to search: Demo Guest
NAME         EMAIL                      RSVP  FOOD
Demo Guest   demo.guest@example.com     yes   veg
```

Additional functionality

- View guests: sort records by name and display in columns.
- Update RSVP: revise food and attendance via a temporary file and atomic replacement.
- Backup: archive data and invitation files with tar -czf.
- Activity log: record timestamped operations and show latest entries.

The event name, actual date, time, and venue must be edited in data/event.conf before producing final invitations.

AUTOMATION / VERIFICATION

Cron, testing and submission

Scheduled report at 6:00 PM

Running ./main.sh --cron saves the latest yes/no/maybe counts to reports/rsvp_status.txt. The script can stop generating new reports after EVENT_END_DATE is set; remove the cron entry after the event.

```
0 18 * * * /bin/bash $HOME/linux_project/rsvpdesk/main.sh --cron  >> $HOME/linux_project/rsvpdesk/logs/cron.log 2>&1

$ ./main.sh --cron
RSVP status saved: reports/rsvp_status.txt
```

Functional verification

The included tests/run_tests.sh tests on a temporary copy of the project to avoid changing the original guest list. It checks Bash syntax, initial data, summaries, validation, searching, invitation files, updates, backup, report generation, and event date cutoff.

```
PASS: Bash syntax
PASS: At least 30 fictional guests
PASS: Adding a validated guest / duplicate rejected
PASS: Here-document invitation generation
PASS: Backup archive and contents
PASS: Cron report generated / event end date honored
ALL 14 CHECKS PASSED
```

Submission requirements and evidence

- Submit F48_L37_rsvpdesk.tar.gz containing source code, data, documentation and relevant folders.
- Take real terminal screenshots of all five required features, menu, backup and activity log.
- Capture crontab -l, reports/rsvp_status.txt, and logs/cron.log after a real scheduled execution.
- Run the project in Ubuntu/WSL and explain how read, case, awk, grep, sort, uniq, here-doc and tar work.

Learning outcomes / viva preparation

Key topics: shebang (#!/usr/bin/env bash), chmod +x, while/case menus, functions and local variables, CSV parsing, output redirection (> vs >>), $1 and $?, safe filenames, compressed archives, cron time fields, and shell debugging with bash -x.

Evidence note: Actual 6 PM cron execution and screenshots from the student's machine are still required. The tests here demonstrate the command, not a scheduled run.